1. **Задание**

   1 Определите перечень Rest-сервисов, выполняющих те же действия, что и в лабораторной работе 1. Внимательно отнеситесь к вопросу какой HTTP метод использует тот или иной сервис и какие коды HTTP он может возвращать.

   2 Опишите Ваш API с помощью OpenAPI v3. Приведите Ваш yaml файл в отчете.

   3 Средствами swagger сгенерируйте сервер spring.

   4 Возьмите REST контроллер из сгенерированного сервера, вставьте в свое приложение (не забыв поправить имя пакета).

   5 Реализуйте приложение.

2. **Описание продукта**

   Необходимо разработать REST API сервис, позволяющий добавлять, удалять и просматривать список литературы.

3. **Листинг программного кода**

   GOST.yaml

```
openapi: 3.0.1
info:
 title: ГОСТ Литература
 description: Хранение литературы по ГОСТ
 version: 1.0.0
paths:
  /gost:
    post:
      tags:
        - API
      summary: Добавить новый источник
      operationId: AddBook
      requestBody:
        description: Добавление новой литературы
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/GOST'
        required: true
      responses:
        '400':
```

```
      description: Неверный формат
    '404':
      description: Ресурс не найден
    '409':
      description: Данные конфликтуют
    '200':
      description: Успешная операция
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/GOST'
  get:
    tags:
      - API
    summary: Получить все источники
    operationId: GetBooks
    responses:
      '404':
        description: Ресурс не найден
      '200':
        description: Успешная операция
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/GOST'
  delete:
    tags:
      - API
    summary: Удалить источник по идентификатору
    operationId: DeleteBook
    responses:
      '404':
        description: Ресурс не найден
```

```
        '406':
          description: Данных не существует
        '200':
          description: Успешная операция
      parameters:
      - name: id
        in: query
        description: идентификатор литературы
        required: true
        schema:
          type: integer
components:
 schemas:
  GOST:
    type: object
    description: объект, представляющий данные о добавленной литературе
    properties:
      id:
        type: number
        format: long
        description: Идентификатор
      year:
        type: number
        format: long
        description: Год выпуска
      author:
        type: string
        description: Автор
      name:
        type: string
        description: Название
```

GostApi.java

```
@Generated(value = "org.openapitools.codegen.languages.SpringCodegen", date =
"2025-09-21T17:15:00.750982+03:00[Europe/Moscow]", comments = "Generator
```

```
version: 7.7.0")
@Validated
@Tag(name = "API", description = "the API API")
public interface GostApi {

    default Optional<NativeWebRequest> getRequest() {
        return Optional.empty();
    }

    /**
     * POST /gost : Добавить новый источник
     *
     * @param GOST Описание метода (required)
     * @return Неверный формат (status code 400)
     *         or Ресурс не найден (status code 404)
     *         or Данные конфликтуют (status code 409)
     *         or Успешная операция (status code 200)
     */
    @Operation(
        operationId = "addBook",
        summary = "Добавить новый источник",
        tags = { "API" },
        responses = {
            @ApiResponse(responseCode = "400", description = "Неверный формат"),
            @ApiResponse(responseCode = "404", description = "Ресурс не найден"),
            @ApiResponse(responseCode = "409", description = "Данные конфликтуют"),
            @ApiResponse(responseCode = "200", description = "Успешная операция",
content = {
                    @Content(mediaType = "application/json", schema =
@Schema(implementation = GOST.class))
            })
        }
    )
    @RequestMapping(
        method = RequestMethod.POST,
```

```
        value = "/gost",
        produces = { "application/json" },
        consumes = { "application/json" }
    )

    ResponseEntity<GOST> addBook(@Parameter(name = "GOST", description =
"Описание метода", required = true) @Valid @RequestBody GOST GOSTBody);

    /**
     * DELETE /gost : Удалить источник по идентификатору
     *
     * @param id идентификатор факультета (required)
     * @return Ресурс не найден (status code 404)
     *         or Данных не существует (status code 406)
     *         or Успешная операция (status code 200)
     */
    @Operation(
        operationId = "deleteBook",
        summary = "Удалить источник по идентификатору",
        tags = { "API" },
        responses = {
            @ApiResponse(responseCode = "404", description = "Ресурс не найден"),
            @ApiResponse(responseCode = "406", description = "Данных не существует"),
            @ApiResponse(responseCode = "200", description = "Успешная операция")
        }
    )
    @RequestMapping(
        method = RequestMethod.DELETE,
        value = "/gost"
    )

    ResponseEntity<Void> deleteBook(@NotNull @Parameter(name = "id", description =
"идентификатор факультета", required = true, in = ParameterIn.QUERY) @Valid
@RequestParam(value = "id", required = true) Long id);
    /**
```

```
    * GET /gost : Получить все источники
    *
    * @return Ресурс не найден (status code 404)
    *         or Успешная операция (status code 200)
    */
   @Operation(
      operationId = "getBooks",
      summary = "Получить все источники",
      tags = { "API" },
      responses = {
         @ApiResponse(responseCode = "404", description = "Ресурс не найден"),
         @ApiResponse(responseCode = "200", description = "Успешная операция",
content = {
            @Content(mediaType = "application/json", array = @ArraySchema(schema =
@Schema(implementation = GOST.class)))
         })
      }
   )
   @RequestMapping(
      method = RequestMethod.GET,
      value = "/gost",
      produces = { "application/json" }
   )

   ResponseEntity<List<GOST>> getBooks();
}
```

GostApiController.java

```
@Generated(value = "org.openapitools.codegen.languages.SpringCodegen", date =
"2025-09-21T17:15:00.750982+03:00[Europe/Moscow]", comments = "Generator
version: 7.7.0")
@Controller
@RequestMapping("${openapi..base-path:}")
@RequiredArgsConstructor
public class GostApiController implements GostApi {
```

```
    private final NativeWebRequest request;

    private final GostWorker GostWorker;

    @Override
    public Optional<NativeWebRequest> getRequest() {
        return Optional.ofNullable(request);
    }

    @Override
    public ResponseEntity<GOST> addBook(@Parameter(name = "GOST", description =
"Добавление новой литературы", required = true) @Valid @RequestBody GOST
GOSTBody)
    {
        if (GostWorker.IsGOSTExist(GOSTBody.getId())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }

        GostWorker.AddGOST(GOSTBody.getId(), GOSTBody);
        return ResponseEntity.ok(GOSTBody);
    }

    @Override
    public ResponseEntity<Void> deleteBook(@NotNull @Parameter(name = "id",
description = "идентификатор литературы", required = true, in = ParameterIn.QUERY)
@Valid @RequestParam(value = "id", required = true) Long id)
    {
        if (!GostWorker.IsGOSTExist(id)) {
            return ResponseEntity.status(HttpStatus.NOT_ACCEPTABLE).build();
        }

        GostWorker.DeleteById(id);
        return null;
    }
```

```
    @Override
    public ResponseEntity<List<GOST>> getBooks()
    {
        return ResponseEntity.ok(GostWorker.ShowAll());
    }
}
```

Gost.java

```
@Schema(name = "GOST", description = "объект, представляющий данные о добавленной литературе")
@Generated(value = "org.openapitools.codegen.languages.SpringCodegen", date = "2025-09-21T17:15:00.750982+03:00[Europe/Moscow]", comments = "Generator version: 7.7.0")
public class GOST {

  private Long id;

  private Long year;

  private String author;

  private String name;

  /**
   * Идентификатор
   * @return id
   */

  @Schema(name = "id", description = "Идентификатор", requiredMode = Schema.RequiredMode.NOT_REQUIRED)
  @JsonProperty("id")
  public Long getId() {
    return id;
  }
```

```
  /**
   * Год выпуска
   * @return year
   */

  @Schema(name = "year", description = "Год выпуска", requiredMode =
Schema.RequiredMode.NOT_REQUIRED)
  @JsonProperty("year")
  public Long getYear() {
    return year;
  }

  /**
   * Автор
   * @return author
   */

  @Schema(name = "author", description = "Автор", requiredMode =
Schema.RequiredMode.NOT_REQUIRED)
  @JsonProperty("author")
  public String getAuthor() {
    return author;
  }

  /**
   * Название
   * @return name
   */

  @Schema(name = "name", description = "Название", requiredMode =
Schema.RequiredMode.NOT_REQUIRED)
  @JsonProperty("name")
  public String getName() {
    return name;
```

```
    }


}
```

GostWorker.java

```
@Service
public class GostWorker {

    private final Map<Long, GOST> GOSTList = new HashMap<>();

    public boolean IsGOSTExist(Long id)
    {
        return GOSTList.containsKey(id);
    }

    public void AddGOST(Long id, GOST GOSTObject)
    {
        GOSTList.put(id, GOSTObject);
    }
    public void DeleteById(Long id)
    {
        GOSTList.remove(id);
    }
    public List<GOST> ShowAll()
    {
        return new ArrayList<>(GOSTList.values());
    }
}
```

4. **Выводы**

В процессе лабораторной работы были получены практические знания разработки REST/JSON сервиса